

SQL QUICK REFERENCE

SQL Cheat Sheet

Complete Interview Reference Card

Aggregate Functions • JOINS • Window Functions • Execution Order
• Most Asked Queries • Common Errors • 1-Page Syntax Summary

Interview Ready | Quick Revision | MySQL 8.0

Extracted from SQL Interview Handbook — Edition 2026

SQL Execution Order

Memory Trick: "From Where Groups Have Selected Distinct Order Limits"

F

ROM →

W

HERE →

G

ROUP BY →

H

AVING →

S

ELECT →

D

ISTINCT →

O

RDER BY →

L

IMIT

Step	Clause	What it does
1	FROM / JOIN	Identifies tables and joins them
2	WHERE	Filters individual rows (before grouping)
3	GROUP BY	Groups remaining rows by column(s)
4	HAVING	Filters groups (after aggregation)
5	SELECT	Picks columns, computes expressions & aliases
6	DISTINCT	Removes duplicate rows
7	ORDER BY	Sorts the final result set
8	LIMIT / OFFSET	Restricts number of rows returned

Key Rule

You CANNOT use a SELECT alias in WHERE or HAVING — because SELECT runs after both of them!

Aggregate Functions Cheat Sheet

Aggregate = Many rows → One result. Always used with GROUP BY or alone.

Function	Returns	NULL Handling	Example
<code>COUNT(*)</code>	Total rows (including NULLs)	Counts NULLs	<code>COUNT(*)</code> → 12
<code>COUNT(col)</code>	Non-NULL rows in column	Skips NULLs	<code>COUNT(rating)</code> → 10
<code>SUM(col)</code>	Total of all values	Ignores NULLs	<code>SUM(price)</code> → 150000
<code>AVG(col)</code>	Average of non-NULL values	Ignores NULLs	<code>AVG(price)</code> → 12500
<code>MIN(col)</code>	Smallest value	Ignores NULLs	<code>MIN(price)</code> → 80
<code>MAX(col)</code>	Largest value	Ignores NULLs	<code>MAX(price)</code> → 65000
<code>GROUP_CONCAT(col)</code>	Joins values into one string	Ignores NULLs	'Laptop,Phone,Chair'

```
SELECT category,
       COUNT(*) AS total_orders,
       SUM(quantity * price_per_unit) AS revenue,
       AVG(price_per_unit) AS avg_price,
       MIN(price_per_unit) AS cheapest,
       MAX(price_per_unit) AS costliest
FROM orders
GROUP BY category
HAVING COUNT(*) > 1
ORDER BY revenue DESC;
```

Common Mistake

`COUNT(*) - COUNT(col)` = number of NULLs in that column. Smart trick!

JOIN Cheat Sheet

JOIN Type	Returns	NULL Fill?
INNER JOIN	Only rows that match in BOTH tables	No
LEFT JOIN	All rows from LEFT table + matched right rows	Right side = NULL if no match
RIGHT JOIN	All rows from RIGHT table + matched left rows	Left side = NULL if no match
CROSS JOIN	Every row of A paired with every row of B	No (Cartesian product)
SELF JOIN	Table joined to itself (use aliases)	Depends
FULL OUTER (MySQL)	LEFT JOIN UNION RIGHT JOIN	Both sides

```
-- INNER JOIN
SELECT o.order_id, o.product, s.seller_name
FROM orders o INNER JOIN sellers s ON o.seller_id = s.seller_id;

-- LEFT JOIN: find orders with NO seller
SELECT o.order_id, s.seller_name
FROM orders o LEFT JOIN sellers s ON o.seller_id = s.seller_id
WHERE s.seller_id IS NULL;    -- unmatched rows only

-- SELF JOIN: employee and their manager
SELECT e.name AS employee, m.name AS manager
FROM employees e LEFT JOIN employees m ON e.manager_id = m.emp_id;

-- FULL OUTER JOIN simulation
SELECT * FROM orders o LEFT JOIN sellers s ON o.seller_id = s.seller_id
UNION
SELECT * FROM orders o RIGHT JOIN sellers s ON o.seller_id = s.seller_id;
```

Memory Trick

LEFT JOIN = "Keep my left side no matter what." RIGHT JOIN = "Keep the right side no matter what."

Window Functions Cheat Sheet

Window Function = does calculation across rows **WITHOUT** collapsing them (unlike **GROUP BY**).

Syntax: `FUNCTION() OVER (PARTITION BY col ORDER BY col)`

Function	Use Case	Ties?
<code>ROW_NUMBER()</code>	Unique sequential number for every row	No ties, always unique
<code>RANK()</code>	Rank with gaps (1,1,3)	Same rank for ties, skips next
<code>DENSE_RANK()</code>	Rank without gaps (1,1,2)	Same rank, no gap
<code>SUM() OVER()</code>	Running total	—
<code>AVG() OVER()</code>	Moving / group average	—
<code>COUNT() OVER()</code>	Running count	—
<code>LAG(col, n)</code>	Value from n rows before current	—
<code>LEAD(col, n)</code>	Value from n rows after current	—
<code>FIRST_VALUE(col)</code>	First value in window partition	—
<code>LAST_VALUE(col)</code>	Last value in window partition	—
<code>NTILE(n)</code>	Divides rows into n equal buckets	—

```
-- Rank products by price within each category
SELECT category, product, price_per_unit,
       RANK() OVER (PARTITION BY category ORDER BY price_per_unit DESC) AS rnk,
       DENSE_RANK() OVER (PARTITION BY category ORDER BY price_per_unit DESC) AS dense_rnk,
       ROW_NUMBER() OVER (PARTITION BY category ORDER BY price_per_unit DESC) AS row_num
FROM orders;

-- Running total of revenue
SELECT order_id, order_date, price_per_unit,
       SUM(price_per_unit) OVER (ORDER BY order_date) AS running_total
FROM orders;

-- Compare current order price with previous (LAG)
SELECT order_id, price_per_unit,
       LAG(price_per_unit, 1) OVER (ORDER BY order_date) AS prev_price,
       price_per_unit - LAG(price_per_unit,1) OVER (ORDER BY order_date) AS diff
FROM orders;

-- Divide orders into 4 price quartiles
SELECT order_id, price_per_unit,
       NTILE(4) OVER (ORDER BY price_per_unit) AS quartile
FROM orders;
```

Most Asked SQL Queries in Interviews

```
-- 1. Nth Highest Value
SELECT DISTINCT price_per_unit FROM orders
ORDER BY price_per_unit DESC LIMIT 1 OFFSET N-1; -- replace N with 2, 3, etc.
-- Using DENSE_RANK (handles ties better):
SELECT price_per_unit FROM (
    SELECT price_per_unit, DENSE_RANK() OVER (ORDER BY price_per_unit DESC) AS dr
    FROM orders) t WHERE dr = N;

-- 2. Find Duplicates
SELECT customer_name, COUNT(*) AS cnt
FROM orders GROUP BY customer_name HAVING COUNT(*) > 1;

-- 3. Delete Duplicates (keep lowest id)
DELETE FROM orders WHERE order_id NOT IN
    (SELECT MIN(order_id) FROM orders GROUP BY customer_name);

-- 4. Rows in Table A but NOT in Table B
SELECT * FROM A LEFT JOIN B ON A.id = B.id WHERE B.id IS NULL;

-- 5. Running Total
SELECT order_id, SUM(price_per_unit) OVER (ORDER BY order_date) FROM orders;

-- 6. Count NULLs in a column
SELECT COUNT(*) - COUNT(delivery_date) AS null_count FROM orders;

-- 7. Pivot using CASE
SELECT city,
    SUM(CASE WHEN category='Electronics' THEN price_per_unit ELSE 0 END) AS electronics,
    SUM(CASE WHEN category='Furniture' THEN price_per_unit ELSE 0 END) AS furniture
FROM orders GROUP BY city;

-- 8. Month-wise Revenue
SELECT YEAR(order_date) yr, MONTH(order_date) mo,
    SUM(quantity * price_per_unit) AS revenue
FROM orders GROUP BY yr, mo ORDER BY yr, mo;

-- 9. Percentage share per group
SELECT category,
    SUM(price_per_unit) AS cat_total,
    SUM(price_per_unit) * 100.0 / SUM(SUM(price_per_unit)) OVER() AS pct
FROM orders GROUP BY category;

-- 10. Orders above category average
SELECT * FROM orders o
WHERE price_per_unit > (
    SELECT AVG(price_per_unit) FROM orders WHERE category = o.category);
```

Common Errors & Fixes

Error Message	Cause	Fix
No database selected	Forgot USE dbname	Run <code>USE ecom;</code>
Unknown column in WHERE / ORDER BY	Using SELECT alias too early	Repeat the expression or use subquery
Duplicate entry for key PRIMARY	Inserting duplicate PK	Remove id from INSERT or use AUTO_INCREMENT
FK constraint fails	Child FK value not in parent	Insert parent row first
Data too long for column	VARCHAR too small	ALTER MODIFY column to larger size
You are using safe update mode	UPDATE/DELETE without WHERE	Add WHERE or <code>SET SQL_SAFE_UPDATES=0</code>
Every derived table must have alias	Subquery in FROM lacks alias	Add <code>AS t</code> after closing <code>)</code>
Incorrect datetime value	Wrong date format	Use <code>'YYYY-MM-DD'</code> format
Can't DROP; check FK constraints	Table referenced by another table	Drop child table first or use CASCADE
Subquery returns more than 1 row	Used <code>=</code> instead of <code>IN</code>	Replace <code>=</code> with <code>IN</code> in WHERE clause

SQL Syntax Summary — 1-Page Revision

Command	Syntax
Create DB	<code>CREATE DATABASE name;</code>
Use DB	<code>USE name;</code>
Create Table	<code>CREATE TABLE t (col type constraint, ...);</code>
Insert	<code>INSERT INTO t (cols) VALUES (vals), (...);</code>
Select	<code>SELECT cols FROM t WHERE cond ORDER BY col LIMIT n;</code>
Update	<code>UPDATE t SET col=val WHERE cond;</code>
Delete	<code>DELETE FROM t WHERE cond;</code>
Group + Filter	<code>SELECT col, AGG() FROM t GROUP BY col HAVING cond;</code>
Join	<code>SELECT ... FROM A [INNER/LEFT/RIGHT] JOIN B ON A.id=B.id;</code>
Subquery	<code>SELECT ... WHERE col IN (SELECT col FROM t WHERE ...);</code>
Union	<code>SELECT ... UNION [ALL] SELECT ...;</code>
Window Function	<code>FUNC() OVER (PARTITION BY col ORDER BY col)</code>
Create View	<code>CREATE VIEW v AS SELECT ...;</code>
Create Index	<code>CREATE INDEX idx ON t(col);</code>
Stored Procedure	<code>DELIMITER // CREATE PROCEDURE p() BEGIN ... END // DELIMITER ;</code>
Trigger	<code>CREATE TRIGGER t AFTER UPDATE ON tbl FOR EACH ROW BEGIN ... END</code>
Transaction	<code>SET autocommit=0; ...changes... COMMIT; / ROLLBACK;</code>
Alter Table	<code>ALTER TABLE t ADD/MODIFY/DROP/RENAME COLUMN col ...;</code>
Drop Table	<code>DROP TABLE [IF EXISTS] t;</code>
Truncate	<code>TRUNCATE TABLE t;</code>
Constraints	<code>PRIMARY KEY, UNIQUE, NOT NULL, CHECK(cond), DEFAULT val</code>
Foreign Key	<code>FOREIGN KEY (col) REFERENCES parent(col) ON DELETE CASCADE</code>
Rollup	<code>GROUP BY col1, col2 WITH ROLLUP</code>
COALESCE	<code>COALESCE(col, default_val)</code> — first non-NULL value
CASE	<code>CASE WHEN cond THEN val ... ELSE val END</code>
Date Functions	<code>YEAR(), MONTH(), DAY(), DATEDIFF(), NOW(), CURRENT_DATE</code>
String Functions	<code>UPPER(), LOWER(), LENGTH(), CONCAT(), TRIM(), SUBSTRING()</code>

Final 5 Golden Rules for Interviews

1. Always use `IS NULL` / `IS NOT NULL` — never `= NULL`.
2. Always add `WHERE` before UPDATE or DELETE (preview with SELECT first).
3. Execution order: FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT.
4. Top-N = `ORDER BY DESC LIMIT N`. Not-in-another-table = `LEFT JOIN ... WHERE B.id IS NULL`.
5. Window functions don't collapse rows — use them instead of subqueries for ranking and running totals.